

C++ Programming

Intern

Name: Amima Batool

Starting Date: September 01, 2026

Ending Date: September 30, 2026

Project no. : Third

Project name: Banking System

Banking System

Introduction:

Banking Management System is a simple command-line application developed using C++. The program allows users to create a customer and bank account, deposit and withdraw money, check account balance, transfer funds, view account information, view transaction history, and save account details to a text file.

The application uses multiple classes, constructors, encapsulation, vectors, functions, conditional statements, loops, and file handling to manage customer, account, and transaction information. This project was developed to practice fundamental C++ programming concepts and object-oriented programming.

Tools & Technologies Used:

- VS Code
- C++
- GitHub

Objectives of the Project:

The main objectives of the Banking System are:

- To develop a simple banking application using C++.
- To practice object-oriented programming concepts.
- To understand and implement classes and objects.
- To learn how to use constructors.
- To practice encapsulation using private and public access specifiers.
- To understand the use of member functions.
- To practice working with vectors.
- To store transaction records using objects.
- To perform deposit and withdrawal operations.
- To calculate and display account balance.
- To implement fund transfer between bank accounts.
- To maintain transaction history.
- To practice file handling for saving account information.
- To gain hands-on experience in developing a command-line application.

Features:

Following are the features of the Banking System:

1. Allows the user to enter Customer ID.

2. Allows the user to enter Customer Name.

3. Allows the user to enter Account Number.

4. Creates a customer object.

5. Creates a bank account object.

6. Allows the user to deposit money.

7. Validates that the deposit amount is positive.

8. Allows the user to withdraw money.

9. Checks whether sufficient balance is available before withdrawal.

10. Allows the user to transfer funds to another account.

11. Validates the transfer amount and available balance.

12. Allows the user to check account balance.

13. Displays customer and account information.

14. Maintains transaction history.

15. Records deposits, withdrawals, and fund transfers.

16. Allows the user to save account details in a text file.

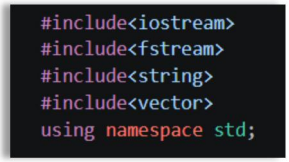
17. Provides a menu-driven command-line interface.

18. Allows the user to exit the application.

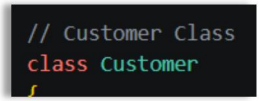
C++ Concepts Used:

Following fundamental C++ Concepts are used in this project:

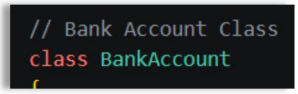
- Header Files and Standard Namespace



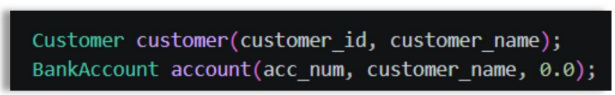
- Classes



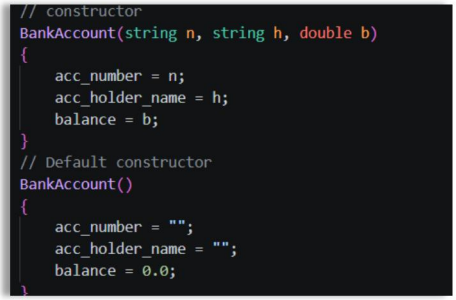




- Objects



- Constructors and Default Constructors



- Parameterized Constructors

```
Customer(string id, string name)
{
    customer_id = id;
    customer_name = name;
}
```

- Encapsulation

```
// Customer Class
class Customer
{
private:
    string customer_id;
    string customer_name;
```

- Private and Public Access Specifiers

```
// Customer Class
class Customer
{
private:
    string customer_id;
    string customer_name;

public:
    Customer()
    {
        customer_id = "";
        customer_name = "";
    }

    Customer(string id, string name)
    {
        customer_id = id;
        customer_name = name;
    }

    void displayCustomer()
    {
        cout << "Customer ID: " << customer_id << endl;
        cout << "Customer Name: " << customer_name << endl;
    }
};
```

- Member Functions

```
// Deposit Function
void deposit(double amount)
{
    if(amount>0)
    {
        balance += amount;
        cout<<"Amount Deposited: "<<amount<<endl;
        cout<<"New Balance: "<< balance<<endl;
        cout<<"Amount Deposited Successfully!"<<endl;
        transactions.push_back(Transaction("Deposit ", amount));
    }
    else
    {
        cout<<"Deposit Amount Must Be Positive. Try Again!";
    }
    cout<<endl;
}
// withdraw Function
void withdraw(double amount)
{
    if(amount>0 && amount<=balance)
    {
        balance -= amount;
        cout<<"Amount Withdraw: "<<amount<<endl;
        cout<<"New Balance: "<< balance<<endl;
        cout<<"Amount Withdraw Successfully!"<<endl;
        transactions.push_back(Transaction("Withdrawal ", amount));
    }
    else
    {
```

- Vectors

```
vector<Transaction> transactions;
```

- "vector::push_back()"

```
transactions.push_back(Transaction("Withdrawal ", amount));
```

- "vector::empty()"

```
if(transactions.empty())
{
    cout << "No transactions found." << endl;
    return;
}
```

- "for" and do loops

```
for(int i = 0; i < transactions.size(); i++)
{
    cout << i + 1 << ". ";
    transactions[i].displayTransaction();
}
```

```
do
{
    cout<< "----MENU----"<<endl;
    cout << "1. Deposit Amount" << endl;
    cout << "2. Withdraw Amount" << endl;
    cout << "3. Check Balance" << endl;
    cout << "4. Transfer Funds" << endl;
```

- "switch" statements and if statements

```
switch(choice)
{
    case 1:
    {
        double amount;
        cout<<"Enter Amount to Deposit: ";
        cin>> amount;
        account.deposit(amount);
```

```
if(file.is_open())
{
    file << acc_number << endl;
    file << acc_holder_name << endl;
    file << balance << endl;
    file.close();
    cout << "Account details saved successfully!" << endl;
}
```

- "const" Member Functions

```
// Check Balance Function
void checkbalance() const
{
    cout<<"Account Balance: "<< balance<<endl;
    cout<<endl;
}
```

- "ofstream"

```
ofstream file("account.txt");

if(file.is_open())
{
    file << acc_number << endl;
    file << acc_holder_name << endl;
    file << balance << endl;
    file.close();
    cout << "Account details saved successfully!" << endl;
}
else
{
    cout << "Error! Unable to save account details." << endl;
}
cout<<endl;
```

- "is_open()"

```
if(file.is_open())
{
    file << acc_number << endl;
    file << acc_holder_name << endl;
    file << balance << endl;
    file.close();
    cout << "Account details saved successfully!" << endl;
}
```

Working of the Program:

The execution process of the program is as follows:

1. Three classes, "Customer", "Transaction", and "BankAccount", are created. The "Customer" class stores customer ID and customer name. The "Transaction" class stores transaction type and transaction amount. The "BankAccount" class stores account number, account holder name, balance, and a vector of transactions.

```
// Customer Class
class Customer
{
```

```
// Transaction Class
class Transaction
{
```

```
// Bank Account Class
class BankAccount
{
```

2. The program starts by displaying the title "Welcome to Bank Application". The user is asked to enter: Customer ID, Customer Name and Account Number.

```
int main()
{
    cout<<"========Welcome to Bank Application========"<<endl;
    cout<<endl;
    string customer_id;
    string customer_name;
    string acc_num;

    cout
    << "Enter Customer ID: ";
    cin >> customer_id;
    cout << "Enter Customer Name: ";
    cin >> customer_name;
    cout<<"Enter Account Number: ";
    cin>>acc_num;
```

3. A "Customer" object and a "BankAccount" object are created using the entered information. The program displays a menu containing eight options:

- Deposit Amount
- Withdraw Amount
- Check Balance
- Transfer Funds
- View Account Information
- View Transaction History
- Save Account
- Exit

A "do-while" loop keeps the menu running until the user selects the Exit option.

```
Customer customer(customer_id, customer_name);
BankAccount account(acc_num, customer_name, 0.0);

int choice;

do
{
    cout<< "----MENU----"<<endl;
    cout << "1. Deposit Amount" << endl;
    cout << "2. Withdraw Amount" << endl;
    cout << "3. Check Balance" << endl;
    cout << "4. Transfer Funds" << endl;
    cout << "5. View Account Information" << endl;
    cout << "6. View Transaction History" << endl;
    cout << "7. Save Account" << endl;
    cout << "8. Exit" << endl;
    cout << endl;
```

4. If the user selects Deposit Amount, the "deposit()" function checks whether the amount is greater than zero. If the amount is valid, it is added to the account balance and a Deposit transaction is stored in the transaction vector.

```
// Deposit Function
void deposit(double amount)
{
    if(amount>0)
    {
        balance += amount;
        cout<<"Amount Deposited: "<<amount<<endl;
        cout<<"New Balance: "<< balance<<endl;
        cout<<"Amount Deposited Successfully!"<<endl;
        transactions.push_back(Transaction("Deposit ", amount));
    }
    else
    {
        cout<<"Deposit Amount Must Be Positive. Try Again!";
    }
    cout<<endl;
}
```

5. If the user selects Withdraw Amount, the "withdraw()" function checks whether the amount is positive and does not exceed the available balance. If the withdrawal is valid, the amount is deducted from the balance and a Withdrawal transaction is added to the transaction history.

```
// withdraw Function
void withdraw(double amount)
{
    if(amount>0 && amount<=balance)
    {
        balance -= amount;
        cout<<"Amount Withdraw: "<<amount<<endl;
        cout<<"New Balance: "<< balance<<endl;
        cout<<"Amount Withdraw Successfully!"<<endl;
        transactions.push_back(Transaction("Withdrawal ", amount));
    }
    else
    {
        cout<<"Invalid Amount!"<<endl;
    }
    cout<<endl;
}
```

6. If the user selects Check Balance, the "checkbalance()" function displays the current account balance. If the user selects Transfer Funds, the program asks for the receiver's account number and transfer amount.

```
// Check Balance Function
void checkbalance() const
{
    cout<<"Account Balance: "<< balance<<endl;
    cout<<endl;
}
```

7. A receiver "BankAccount" object is created and the "transfer()" function transfers the specified amount if sufficient balance is available.

```cpp
// Fund Transfer Function
void transfer(BankAccount &receiver, double amount)
{
    if(amount > 0 && amount <= balance)
    {
        balance -= amount;
        receiver.balance += amount;

        transactions.push_back
        (
            Transaction("Transfer Sent", amount)
        );

        receiver.transactions.push_back
        (
            Transaction("Transfer Received", amount)
        );

        cout << "Amount Transferred: " << amount << endl;
        cout << "Transfer Successful!" << endl;
        cout << "New Balance: " << balance << endl;
    }
    else
    {
        cout << "Invalid Amount or Insufficient Balance!" << endl;
    }
    cout<<endl;
}
```

8. If the user selects View Account Information, customer information and bank account information are displayed.

```cpp
// Display Account Information Function
void displayAccount()
{
    cout << endl;
    cout << "------ACCOUNT INFORMATION------" << endl;
    cout << "Account Number: " << acc_number << endl;
    cout << "Account Holder: " << acc_holder_name << endl;
    cout << "Balance: " << balance << endl;
    cout<<endl;
}
```

9. If the user selects View Transaction History, the program displays all stored transactions using a range of transaction records.

```cpp
// Display Transaction History
void showTransactions()
{
    cout << endl;
    cout << "----- TRANSACTION HISTORY -----" << endl;

    if(transactions.empty())
    {
        cout << "No transactions found." << endl;
        return;
    }

    for(int i = 0; i < transactions.size(); i++)
    {
        cout << i + 1 << ". ";
        transactions[i].displayTransaction();
    }
    cout<<endl;
}
```

10. If the user selects Save Account, the "saveAccount()" function stores the account number, account holder name, and balance in "account.txt".

```cpp
// Save Account Faunction
void saveAccount()
{
    ofstream file("account.txt");

    if(file.is_open())
    {
        file << acc_number << endl;
        file << acc_holder_name << endl;
        file << balance << endl;
        file.close();
        cout << "Account details saved successfully!" << endl;
    }
    else
    {
        cout << "Error! Unable to save account details." << endl;
    }
    cout<<endl;
}
```

11. If the user selects Exit, the program displays an exit message and terminates.

```cpp
        break;
        case 8:
        {
            cout<<"Exiting...."<<endl;
            cout<<"Thank for using Bank Appication"<<endl;
        }
        break;
        default:
        cout<<"Invalid Option! Try Again!";
    }
```

Application Flow:

The overall basic workflow of the application is as follow:

Start → Enter Customer Information → Create Customer & Account → Display Menu → Select Banking Operation → Perform Operation → Update Balance/Transaction History → Display Result → Return to Menu → Exit → END

Code:

```
// BANKING MANAGEMENT SYSTEM
#include<iostream>
#include<fstream>
#include<string>
#include<vector>
using namespace std;
// Customer Class
class Customer
{
private:
    string customer_id;
```

```
    string customer_name;
public:
    Customer()
    {
        customer_id = "";
        customer_name = "";
    }
    Customer(string id, string name)
    {
        customer_id = id;
        customer_name = name;
    }
    void displayCustomer()
    {
        cout << "Customer ID: " << customer_id << endl;
        cout << "Customer Name: " << customer_name << endl;
    }
};
// Transaction Class
class Transaction
{
private:
    string type;
    double amount;
public:
    Transaction(string t, double a)
    {
        type = t;
        amount = a;
    }
    void displayTransaction()
```

```
    {
        cout << type << " : " << amount << endl;
    }
};
// Bank Account Class
class BankAccount
{
    private:
      string acc_number;
      string acc_holder_name;
      double balance;
      vector<Transaction> transactions;
    public:
    // constructor
    BankAccount(string n, string h, double b)
    {
        acc_number = n;
        acc_holder_name = h;
        balance = b;
    }
    // Default constructor
    BankAccount()
    {
        acc_number = "";
        acc_holder_name = "";
        balance = 0.0;
    }
    // Deposit Function
    void deposit(double amount)
    {
        if(amount>0)
```

```
    {
        balance += amount;
        cout<<"Amount Deposited: "<<amount<<endl;
        cout<<"New Balance: "<< balance<<endl;
        cout<<"Amount Deposited Successfully!"<<endl;
        transactions.push_back(Transaction("Deposit ", amount));
    }
    else
    {
        cout<<"Deposit Amount Must Be Positive. Try Again!";
    }
    cout<<endl;
}
// withdraw Function
void withdraw(double amount)
{
    if(amount>0 && amount<=balance)
    {
        balance -= amount;
        cout<<"Amount Withdraw: "<<amount<<endl;
        cout<<"New Balance: "<< balance<<endl;
        cout<<"Amount Withdraw Successfully!"<<endl;
        transactions.push_back(Transaction("Withdrawal ", amount));
    }
    else
    {
        cout<<"Invalid Amount!"<<endl;
    }
    cout<<endl;
}
 // Fund Transfer Function
```

```
void transfer(BankAccount &receiver, double amount)
{
    if(amount > 0 && amount <= balance)
    {
        balance -= amount;
        receiver.balance += amount;
        transactions.push_back
        (
            Transaction("Transfer Sent", amount)
        );
        receiver.transactions.push_back
        (
            Transaction("Transfer Received", amount)
        );
        cout << "Amount Transferred: " << amount << endl;
        cout << "Transfer Successful!" << endl;
        cout << "New Balance: " << balance << endl;
    }
    else
    {
        cout << "Invalid Amount or Insufficient Balance!" << endl;
    }
    cout<<endl;
}
// Check Balance Function
void checkbalance() const
{
    cout<<"Account Balance: "<< balance<<endl;
    cout<<endl;
}
// Display Account Information Function
```

```
void displayAccount()
{
    cout << endl;
    cout << "------ACCOUNT INFORMATION------" << endl;
    cout << "Account Number: " << acc_number << endl;
    cout << "Account Holder: " << acc_holder_name << endl;
    cout << "Balance: " << balance << endl;
    cout<<endl;
}
// Display Transaction History
void showTransactions()
{
    cout << endl;
    cout << "----- TRANSACTION HISTORY -----" << endl;
    if(transactions.empty())
    {
        cout << "No transactions found." << endl;
        return;
    }
    for(int i = 0; i < transactions.size(); i++)
    {
        cout << i + 1 << ". ";
        transactions[i].displayTransaction();
    }
    cout<<endl;
}
// Save Account Faunction
void saveAccount()
{
    ofstream file("account.txt");
```

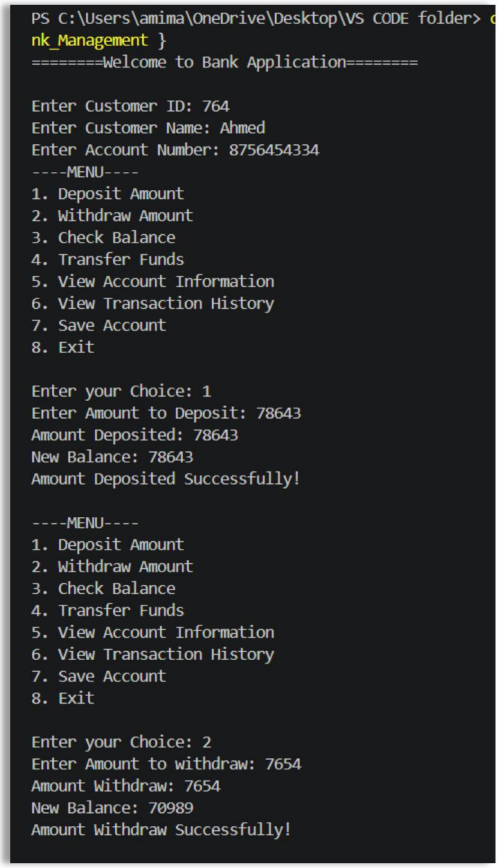
```
        if(file.is_open())
        {
            file << acc_number << endl;
            file << acc_holder_name << endl;
            file << balance << endl;
            file.close();
            cout << "Account details saved successfully!" << endl;
        }
        else
        {
            cout << "Error! Unable to save account details." << endl;
        }
        cout<<endl;
    }
};
int main()
{
    cout<<"========Welcome to Bank Application========"<<endl;
    cout<<endl;
    string customer_id;
    string customer_name;
    string acc_num;
    cout << "Enter Customer ID: ";
    cin >> customer_id;
    cout << "Enter Customer Name: ";
    cin >> customer_name;
    cout<<"Enter Account Number: ";
    cin>>acc_num;
    Customer customer(customer_id, customer_name);
    BankAccount account(acc_num, customer_name, 0.0);
    int choice;
```

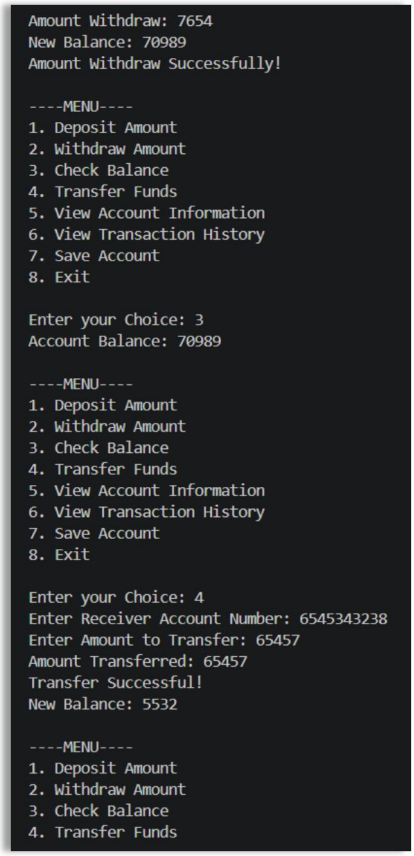
```
do
{
    cout<< "----MENU----"<<endl;
    cout << "1. Deposit Amount" << endl;
    cout << "2. Withdraw Amount" << endl;
    cout << "3. Check Balance" << endl;
    cout << "4. Transfer Funds" << endl;
    cout << "5. View Account Information" << endl;
    cout << "6. View Transaction History" << endl;
    cout << "7. Save Account" << endl;
    cout << "8. Exit" << endl;
    cout << endl;
    cout<<"Enter your Choice: ";
    cin>> choice;
    switch(choice)
    {
        case 1:
        {
            double amount;
            cout<<"Enter Amount to Deposit: ";
            cin>> amount;
            account.deposit(amount);
        }
        break;
        case 2:
        {
            double amount;
            cout<<"Enter Amount to withdraw: ";
            cin>> amount;
            account.withdraw(amount);
        }
```

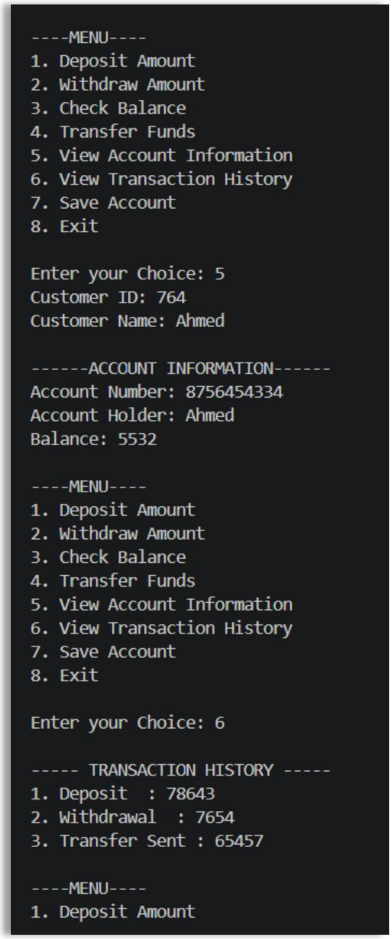
```
        break;
        case 3:
        {
            account.checkbalance();
        }
        break;
        case 4:
        {
            string receiver_num;
            double amount;
            cout << "Enter Receiver Account Number: ";
            cin >> receiver_num;
            cout << "Enter Amount to Transfer: ";
            cin >> amount;
            BankAccount receiver(receiver_num, "Receiver", 0.0);
            account.transfer(receiver, amount);
        }
        break;
        case 5:
        {
            customer.displayCustomer();
            account.displayAccount();
        }
        break;
        case 6:
        {
            account.showTransactions();
        }
        break;
        case 7:
        {
```

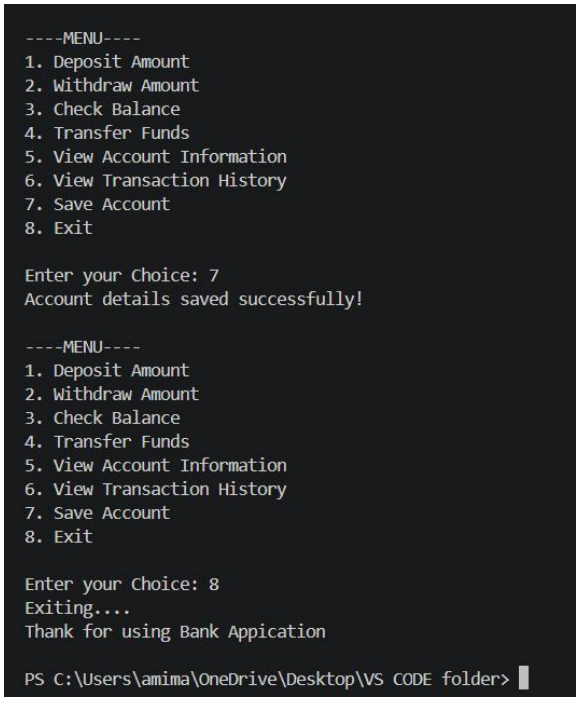
```
            account.saveAccount();
        }
        break;
        case 8:
        {
            cout<<"Exiting...."<<endl;
            cout<<"Thank for using Bank Appication"<<endl;
        }
        break;
        default:
        cout<<"Invalid Option! Try Again!";
    }
  }while(choice != 8);
  cout<<endl;
  return 0;
}
```

Sample Output:









Testing:

The application was tested by performing each operation:

Sr.	Task	Result	Status
1.	Customer information	Customer ID and name are accepted correctly	Passed
2.	Account creation	Account information is created correctly.	Passed
3.	Deposit	Valid deposit amount is added to the balance	Passed
4.	Withdrawal	Valid withdrawal amount is deducted correctly	Passed
5.	Check balance	Current account balance is displayed correctly	Passed
6.	Fund transfer	Valid amount is transferred successfully	Passed
7.	Transaction history	Deposit, withdrawal, and transfer records are displayed.	Passed
8.	Account information	Customer and account details are displayed correctly	Passed
9.	Save account	Account information is successfully saved.	Passed
10.	Exit	Program exits successfully	Passed

Conclusion:

The Banking Management System successfully demonstrates the practical use of fundamental C++ and object-oriented programming concepts. The project provides basic banking operations such as deposit, withdrawal, balance checking, fund transfer, account information display, transaction history, and account data saving.

The application provides a simple command-line interface for managing basic banking operations and demonstrates how object-oriented programming can be applied to a real-world inspired problem.

GitHub Repository:

AmimaBatool/CodeAlpha_Bank-System: A simple C++ console-based banking system for managing customers, bank accounts, deposits, withdrawals and transactions using file handlings.

Project status: Completed.